EXPERIMENT NO: 7 **Date:** _____

SORTING

AIM

Write a program to Sort the elements of a list using function

OBJECTIVES

- To define modular functions accepting list parameters in Python.
- To implement bubble sort algorithm using nested loops and element swapping.
- To return and print ordered data structures.

ALGORITHM

Step 1 :- Initialize the bubble sort program to rearrange list elements in non decreasing numerical order.

Step 2 :- Declare bubble sort function taking list argument arr to perform element sorting operations.

Step 3 :- Calculate total element count of input list arr and store integer length in n.

Step 4 :- Execute outer loop for variable i ranging from index zero to length n minus one.

Step 5 :- Execute inner loop for variable j ranging from index zero to n minus i minus one.

Step 6 :- Check if element at index j is strictly greater than adjacent element at j plus one.

Step 7 :- Swap adjacent elements in place if left element is larger than right element.

Step 8 :- Return ordered list arr from bubble sort function after completing all comparison passes.

Step 9 :- Call sorting function with sample list and print sorted result to terminal window.

Step 10 :- Conclude execution of sorting algorithm program and free list memory allocation from system.

PROGRAM

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

sample_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = bubble_sort(sample_list)
print(f"Sorted list in non-decreasing order: {sorted_list}")
```

SAMPLE OUTPUT

Terminal Output

```
Sorted list in non-decreasing order:  [11, 12, 22, 25, 34, 64, 90]
```

RESULT

The list sorting function was successfully implemented using bubble sort and output verified.

EXPERIMENT NO: 8 **Date:** _____

MAXIMUM AND MINIMUM ELEMENT OF A TUPLE

AIM

Write a program to find Maximum and minimum elements of a tuple using function

OBJECTIVES

- To explore immutable sequence types (tuples) in Python.
- To traverse tuple elements and perform conditional minimum/maximum evaluation.
- To return tuple pairs containing multiple values from functions.

ALGORITHM

Step 1 :- Initialize program execution to find maximum and minimum elements of a tuple using function.

Step 2 :- Declare function find max min accepting tuple parameter tup for processing element extremes.

Step 3 :- Return None for both maximum and minimum if input tuple tup is empty.

Step 4 :- Set initial variables max element and min element to first element of tup.

Step 5 :- Iterate through each numeric element num present inside input tuple tup in order.

Step 6 :- Compare num with max element and update max element if num is strictly larger.

Step 7 :- Compare num with min element and update min element if num is strictly smaller.

Step 8 :- Return tuple containing calculated max element and min element from custom function.

Step 9 :- Pass sample tuple to function and print maximum and minimum values to terminal screen.

Step 10 :- Conclude tuple analysis program execution successfully and clear tuple structures from memory.

PROGRAM

```
def find_max_min(tup):
    if not tup:
        return None, None
    max_element = tup[0]
    min_element = tup[0]

    for num in tup:
        if num > max_element:
            max_element = num
        if num < min_element:
            min_element = num
    return max_element, min_element

```

```
sample_tuple = (3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5)
max_element, min_element = find_max_min(sample_tuple)
print(f"The maximum element in the tuple is: {max_element}")
print(f"The minimum element in the tuple is: {min_element}")
```

SAMPLE OUTPUT

Terminal Output

```
The maximum element in the tuple is:  9
The minimum element in the tuple is:  1
```

RESULT

The function to find maximum and minimum elements of a tuple was executed and verified.

EXPERIMENT NO: 9 **Date:** _____

LIST OF FREQUENCIES

AIM

Write a program to create list of frequencies in non-increasing order using function

OBJECTIVES

- To build frequency maps using Python dictionaries.
- To extract dictionary values into lists and implement custom sorting logic.
- To arrange frequency counts in descending numerical order.

ALGORITHM

Step 1 :- Initialize frequency distribution analyzer program to count list occurrences and sort in descending order.

Step 2 :- Declare function frequency list accepting array parameter arr for calculating item occurrence counts.

Step 3 :- Return empty list immediately if input list arr contains no numerical elements.

Step 4 :- Iterate items in list arr, updating element counts inside dictionary frequency dict.

Step 5 :- Convert dictionary count values into linear list frequencies for sorting processing.

Step 6 :- Run outer loop over frequency list indices to begin descending bubble sort algorithm.

Step 7 :- Run inner loop comparing current frequency against subsequent elements in frequencies list.

Step 8 :- Swap elements if preceding frequency count is smaller than following frequency count.

Step 9 :- Return sorted frequency list from function and print result to standard output screen.

Step 10 :- Conclude execution of frequency counting script and release dictionary memory back to system.

PROGRAM

```
def frequency_list(arr):
    if not arr:
        return []
    frequency_dict = {}

    for item in arr:
        if item in frequency_dict:
            frequency_dict[item] += 1
        else:
            frequency_dict[item] = 1
    frequencies = list(frequency_dict.values())

    for i in range(len(frequencies)):
```

```
        for j in range(i + 1, len(frequencies)):
            if frequencies[i] < frequencies[j]:
                frequencies[i], frequencies[j] = frequencies[j], frequencies
    [i]

    return frequencies

sample_list = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
frequencies = frequency_list(sample_list)
print(f"List of frequencies in non-increasing order: {frequencies}")
```

SAMPLE OUTPUT

Terminal Output

```
List of frequencies in non-increasing order:  [3, 2, 2, 1, 1, 1, 1]
```

RESULT

The frequency list computation program in non-increasing order was tested and verified.